

CMPT 479/982: GPU Architecture & Programming

Fall 2026

GPU Memory Architecture & Performance Optimizations

Alaa Alameldeen (alaa@sfu.ca)

Recap: Matrix Multiplication Version 1

Courtesy: PMPP Textbook

```
01  __global__ void MatrixMulKernel(float* M, float* N,  
02                                float* P, int width) {  
03      int row = blockIdx.y*blockDim.y+threadIdx.y;  
04      int col = blockIdx.x*blockDim.x+threadIdx.x;  
05      if ((row < width) && (col < width)) {  
06          float Pvalue = 0;  
07          for (int k = 0; k < width; ++k) {  
08              Pvalue += M[row*width+k]*N[k*width+col];  
09          }  
10          P[row*width+col] = Pvalue;  
11      }  
12 }
```

FIGURE 3.11 A matrix multiplication kernel using one thread to compute one P element.

- gridDim: 2x2, blockDim: 2x2. Four threads in each block, and 4 blocks in the grid
- Each thread computing one element of the output matrix based on its thread coordinates

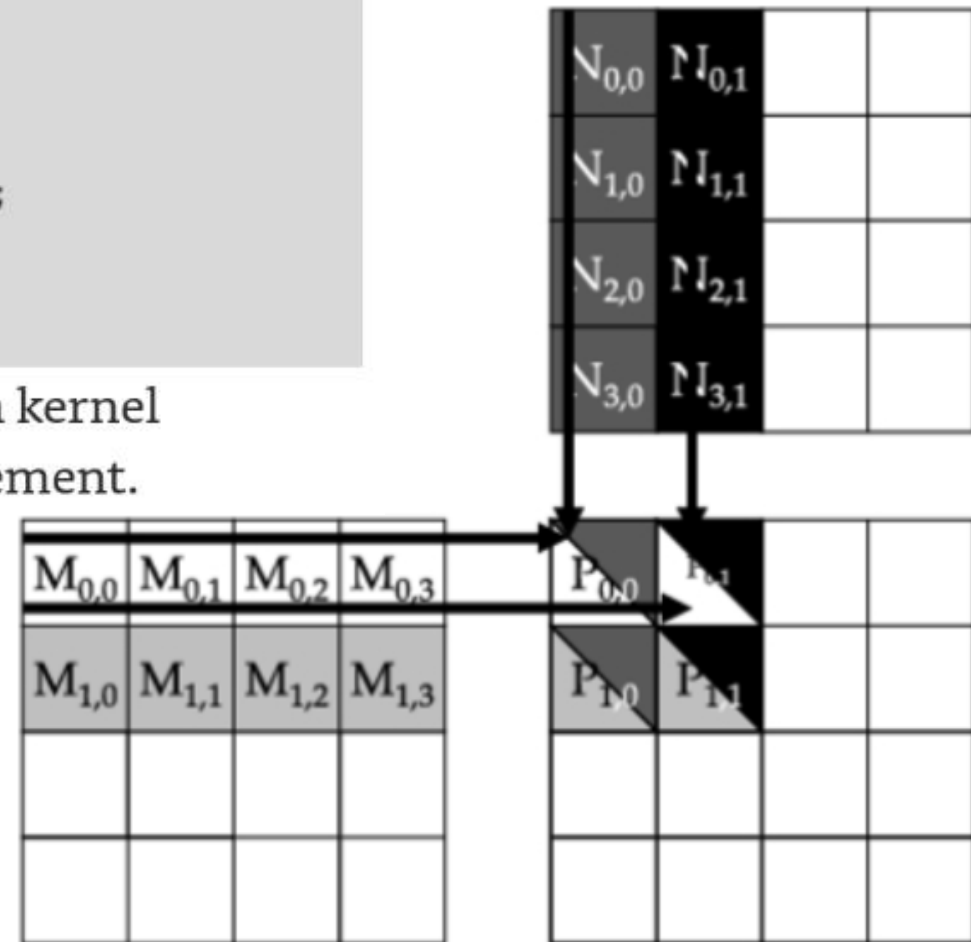


FIGURE 3.13 Matrix multiplication actions of one thread block.

Recap: GPU Compute Architecture

Courtesy: PMPP Textbook

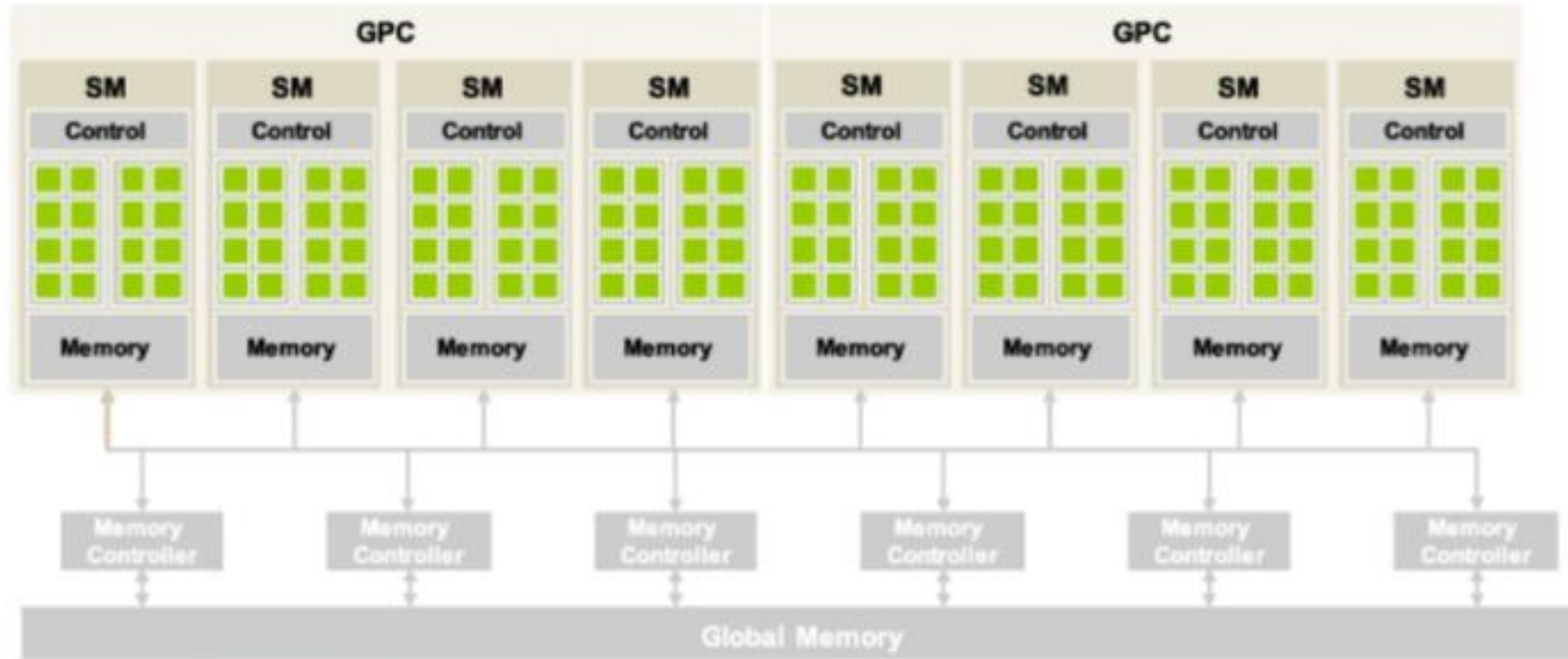


FIGURE 4.1 Architecture of a CUDA-capable GPU.

- GPU includes a few GPU Processing Clusters (GPCs)
- Each GPC has a few Streaming Multiprocessors (SMs). Each SM has a few Streaming Processors (SPs), also known as CUDA cores.
- A few other components include Ray Tracing (RT) cores, Tensor cores, Texture Units, FP64 cores
- On-chip memory resources and global memory with GDDR/HBM memory.

Recap: Warps & Thread Scheduling

Courtesy: PMPP Textbook

- **Zero-overhead scheduling:** Allows fast switching from waiting warp to ready warp: Need more warps per SM
- **Occupancy:** Ratio between #warps assigned to an SM and the maximum number it supports
- **SM Resources:** SPs, registers, shared memory, thread block slots, thread slots
- **H100 SM resources:** 32 blocks, 64 warps (2048 threads), 1024 threads/block, 64K (65,536) registers
- **RTX2000 SM resources:** 24 blocks, 48 warps (1536 threads), 1024 threads/block, 64K registers

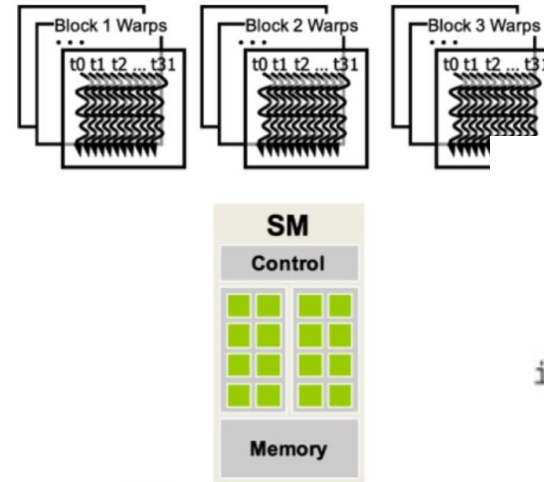


FIGURE 4.6 Blocks are partitioned into warps for thread scheduling.

```
if (threadIdx.x < 24) {  
    A  
}  
else {  
    B  
}  
C
```

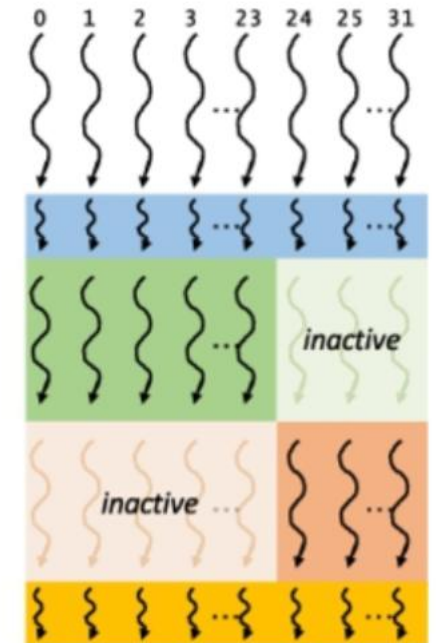


FIGURE 4.9 Example of a warp diverging at an if-else statement.

GPU Memory Architecture

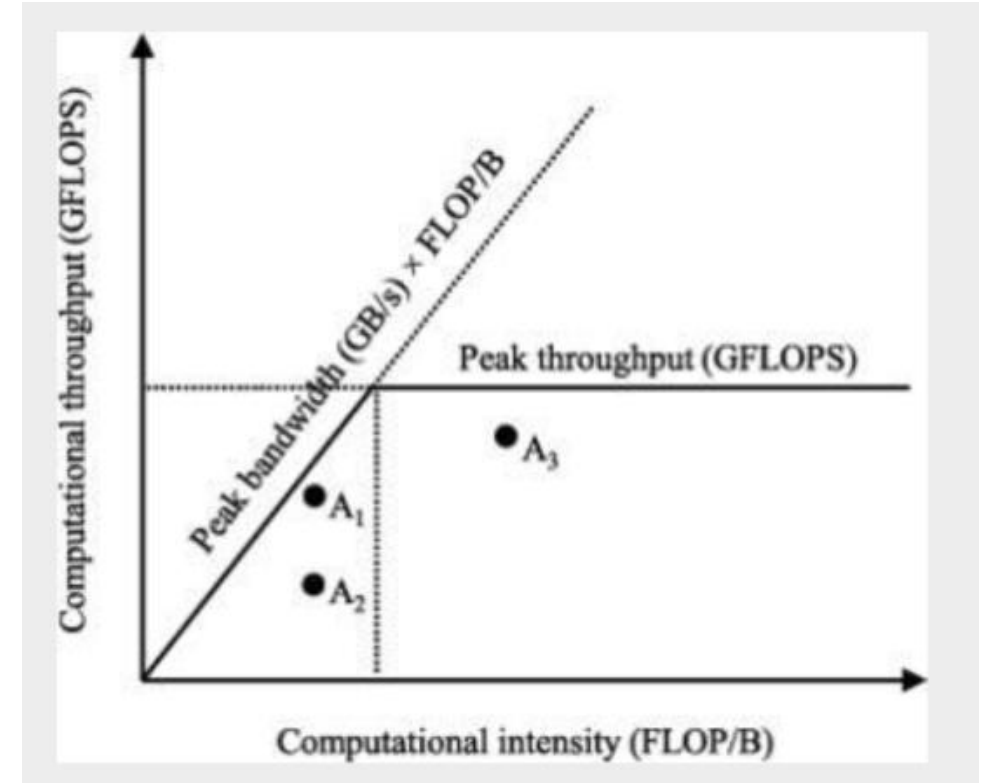
GPU Limits on Performance

- Two main performance limiters:
 1. **Peak Computational Throughput:** Limit on #arithmetic (FP32) operations that GPU hardware can perform per unit time. Example: ~67 TFLOPS for H100, 12 TFLOPS for RTX 2000. Changes based on data type (higher for FP16, lower for FP64).
 2. **Peak Memory Bandwidth:** Limit on #bytes of data can be accessed by hardware from a memory structure per unit time. Example: 3.35 TB/s for H100, 224 GB/sec for RTX 2000
- **Compute-bound vs. Memory-bound kernels:**
 - Compute-bound: Performance is limited by the hardware computational throughput of the hardware. Compute cores are busy most of the time, limited by how fast the cores can execute instructions.
 - Memory-bound: Performance is limited by hardware memory bandwidth. Memory channels are busy most of the time, while compute cores are frequently idle waiting for memory. Performance is limited by how fast memory can supply data.
- **FLOP/B** determines whether a kernel is compute- or memory-bound. Referred to as ***compute-to-global-memory-access ratio***, ***arithmetic intensity*** or ***computational intensity***.
 - High ratio implies compute-bound kernel, low ratio implies memory bound kernel

Roofline Model

Courtesy: PMPP Textbook

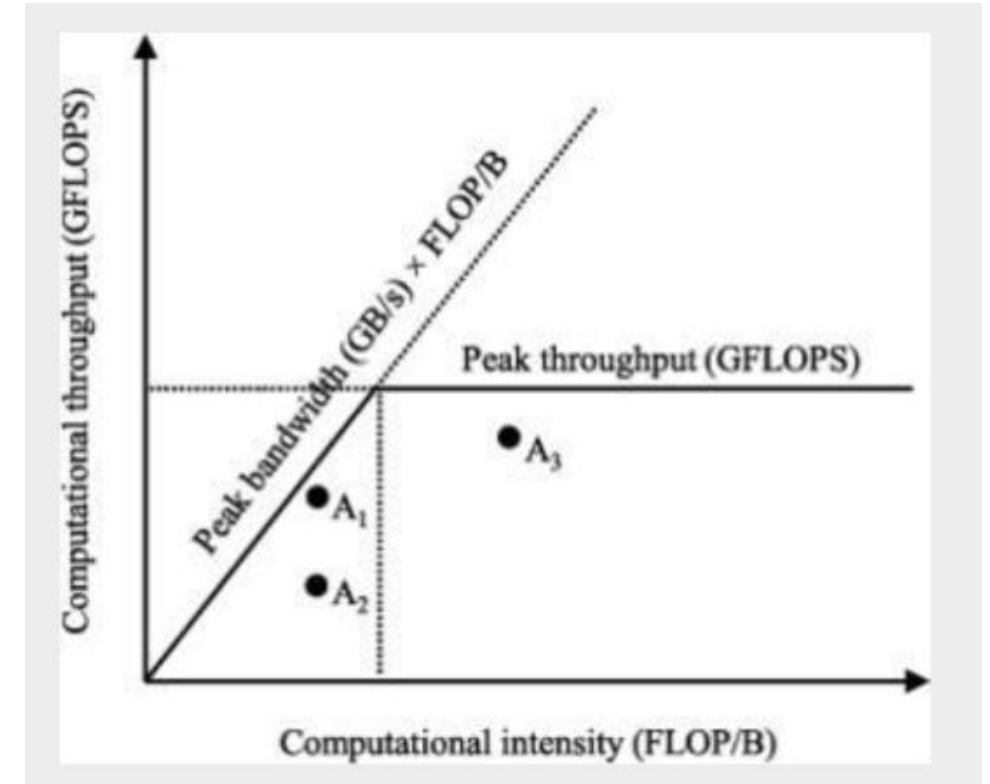
- Used to show how close a kernel is to hardware limits
 - X-axis: FLOP/B (Computational Intensity)
 - Y-axis: GFLOPS (Computational Throughput)
- Horizontal line shows peak throughput (compute limit)
- Diagonal line shows bandwidth limit (Peak BW x FLOP/B)
- Intersection represents the computational intensity threshold at which applications transition from being memory-bound to being compute bound.
- Example: H100 Peak throughput is 67 TFLOPS, Peak BW: 3.35 TB/s. Intersect at $(67 \text{ TFLOPS}) / (3.35 \text{ TB/s}) = 20 \text{ FLOP/B}$
 - A1 and A2 kernels have lower FLOP/B so they are memory-bound. A3 has higher FLOP/B so compute-bound
 - More efficient kernels are closer to hardware limits



Roofline Analysis of Vector Add Kernel

Courtesy: PMPP Textbook

- Each thread performs the same computation:
$$C[i] = A[i] + B[i]$$
- One FP Add, two loads and one store. Each load and store is 4B, so total memory needed is 12B
- Computational Intensity = $1 \text{ FLOP}/12\text{B} = 0.083 \text{ FLOP/B}$
- This is lower than H100's $20 \text{ FLOP/B} \Rightarrow$ Memory-bound
- **Speed-of-Light Analysis** Example:
 - Assume we add two vectors of 1B elements each. Total memory = 3 vectors x 1B elements x 4B = 12GB.
 - Assume kernel finishes in 4 ms.
$$\text{Average BW} = 12\text{GB}/4\text{ms} = 3\text{TB/s}$$
 - BW utilization = $3/3.35$ (peak) = 90% so this kernel uses 90% of available bandwidth (metaphor for hardware limit as a speed-of-light physical limit)
 - This implies there is only room for 10% more perf improvement. Best possible execution time is limited by BW: $12\text{GB}/(3.35\text{TB/s}) = 3.6 \text{ ms}$



Is GEMM Compute-Bound or Memory-Bound?

- Assume we multiply two $N \times N$ matrices. Total memory needed is 3 matrices $\times N^2$ elements $\times 4\text{B/element} = 12N^2$.
 - If $N = 1024$, memory needed is 12MB
- Total computations needed for each of the N^2 elements of the output matrix is a dot product of N elements of first matrix and N elements of second matrix. Dot product needs N multiplies and N additions ($2N$ per output element, $2N^3$ total).
 - If $N=1024$, this is 2GFLOP.
- Computational Intensity = $2N^3/12N^2 = 0.167N$
- For $N=1024$, computational intensity is around 167 FLOP/B, much higher than H100's threshold (20 FLOP/B) $\Rightarrow$ GEMM should be compute-bound for large matrices.

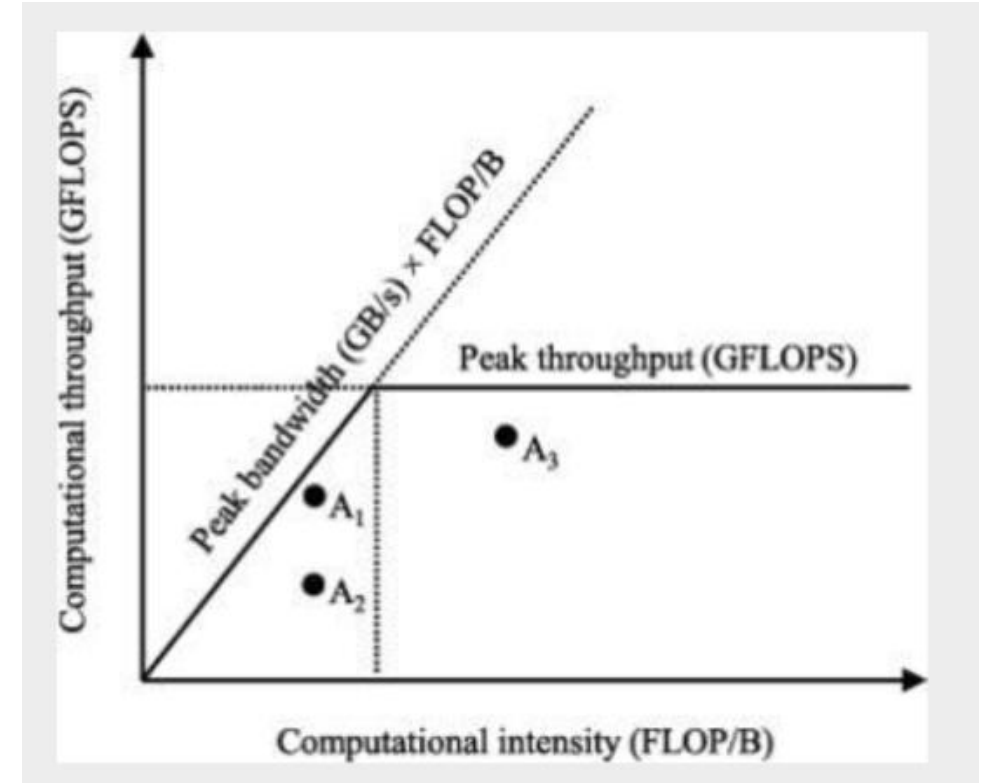
Roofline Analysis of Matrix Multiplication V1

Courtesy: PMPP Textbook

- Dominant part of kernel execution is this for-loop executed in every thread:

```
for (int k = 0; k < Width; ++k) {  
    Pvalue += M[row*Width+k] * N[k*Width+col];  
}
```

- Ignoring the final store of Pvalue, each loop iteration requires reading one element of M, one element of N from memory (8B) and performs one FP32 multiply and one FP32 add (2FLOP).
- Computational Intensity = $2\text{FLOP}/8\text{B} = 0.25 \text{ FLOP/B}$
- Much lower than H100's threshold (20 FLOP/B) $\Rightarrow$ memory-bound
- Also much lower than the $0.167N \text{ FLOP/B}$ in our previous analysis. Why?
- Performance is limited to $(3.35 \text{ TB/s})(0.25 \text{ FLOP/B}) = 0.84 \text{ TFLOPS}$ ($\sim 1.25\%$ of H100's throughput)



GPU Memory Model

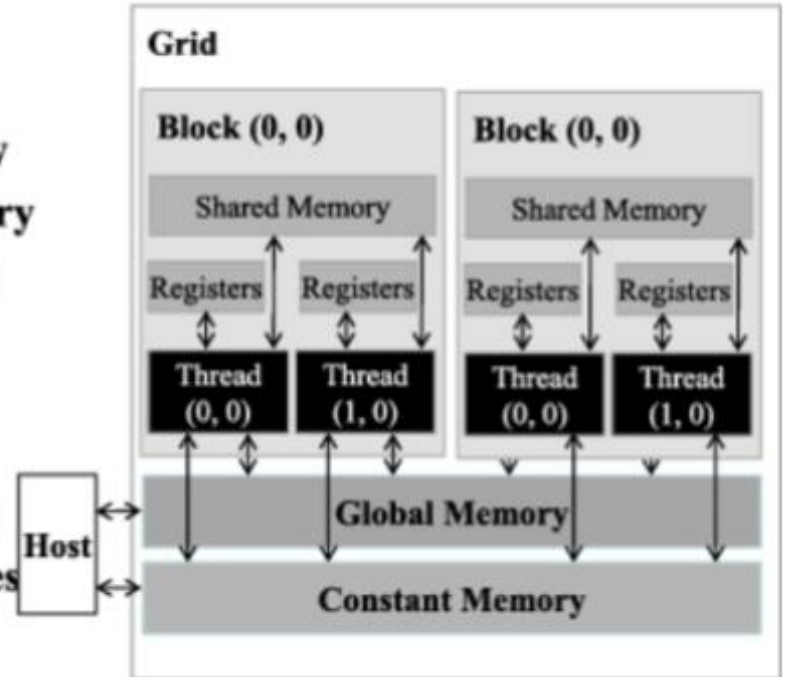
Courtesy: PMPP Textbook

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

Host code can

- Transfer data to/from per grid **global and constant memories**



- Registers are fast, per-thread, used to store automatic local variables.
- Local memory is private, per-thread, located in DRAM (same as global memory). Used for local variables & data that can't be stored in registers (statically allocated arrays, spilled registers, and other elements of the thread's call stack).
- Shared memory is fast and shared between all threads of a block.
- Constant memory supports short-latency, high bandwidth read-only access.

CPU Registers vs. Memory

- Global memory is usually DRAM, off chip with limited bandwidth and slower latency.
- Registers are stored on-chip in much faster storage, with much higher BW than memory.
- Advantages of using registers:
 - Much faster access latency
 - Does not consume memory bandwidth
 - Much lower power and energy vs. memory
 - ALU operations can be performed directly on registers. Memory requires load/store instructions so they increase number of program instructions:

```
load r2, r4, offset // load from memory to r2
fadd r1, r2, r3
```

Courtesy: PMPP Textbook

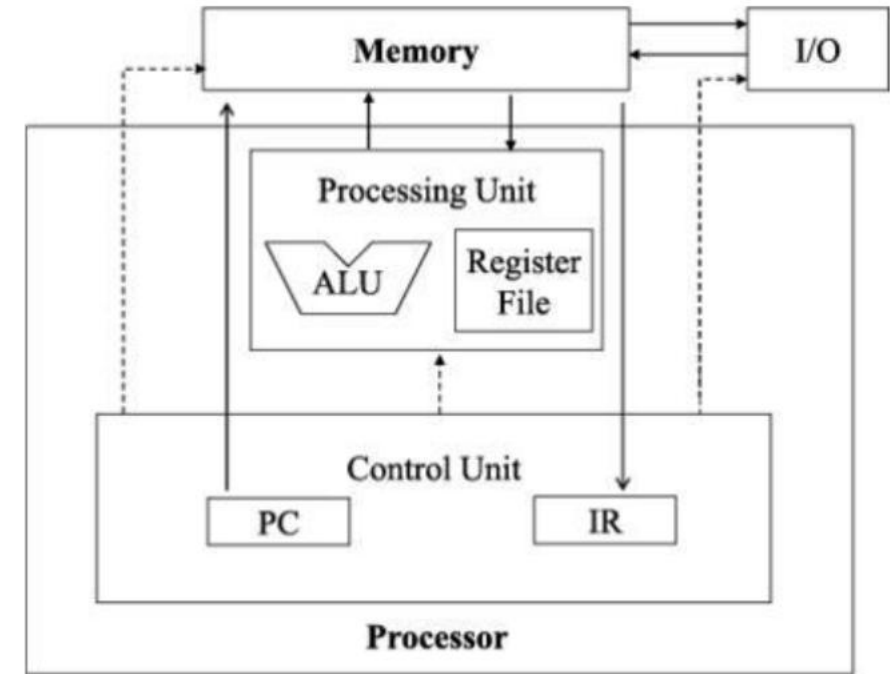


FIGURE 5.2 Memory vs. registers in a modern computer based on the von Neumann model.

GPU Registers vs. Shared Memory

Courtesy: PMPP Textbook

- Shared memory is designed as part of the memory space but is on-chip.
 - Used as a scratchpad memory per block.
 - Accessing data in shared memory requires a memory load operation (similar to global memory).
 - Slower than registers since it requires a load operation
 - Much faster and higher throughput than global memory
- **Distributed shared memory:** Starting in Hopper arch, threads in the same thread block cluster can access shared memory of any block in the cluster

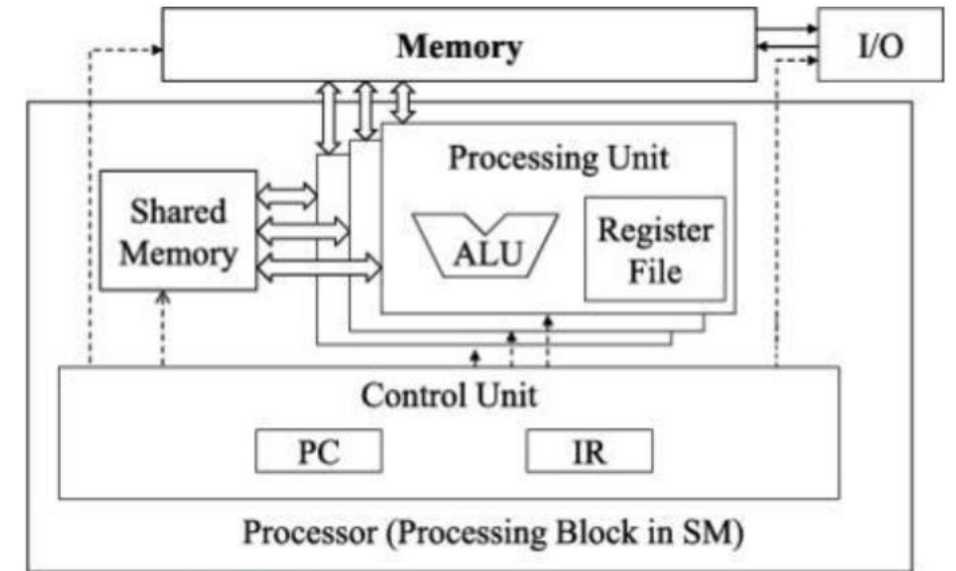


FIGURE 5.3 Shared memory vs. registers in a CUDA device SM.

CUDA Variable Declaration

Courtesy: PMPP Textbook

Variable declaration	Memory	Scope	Lifetime
Automatic variables other than arrays	register	thread	grid
Automatic array variables	local	thread	grid
<code>__device__ __shared__ int SharedVar;</code>	shared	block	grid
<code>__device__ int GlobalVar;</code>	global	grid	application
<code>__device__ __constant__ int ConstVar;</code>	constant	grid	application

FIGURE 5.4 CUDA variable declaration type qualifiers and the properties of each type.

- Scope determines #copies of a variable. A per-thread variable is replicated for all threads in the grid. Per-grid is one copy for the whole kernel
- Lifetime determines how long a variable is accessible. “grid” means values are declared in the kernel body, available for one kernel but don’t persist to subsequent kernel calls. “application” means values persist across kernels in the same application.
- Automatic scalar values are stored in registers. Automatic array variables use local memory (but compiler can optimize and store in registers if index values are constant)

Analyzing Memory Usage of GEMM V1

Courtesy: PMPP Textbook

- Each thread loads one row of M and one column of N from global memory
- Loads from the same row are repeated across all threads processing that row; same for loads from the same column.
- Repeated accesses can be optimized by using tiles, assigning threads processing a tile to the same block

Access order →				
thread _{0,0}	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
thread _{0,1}	$M_{0,0} * N_{0,1}$	$M_{0,1} * N_{1,1}$	$M_{0,2} * N_{2,1}$	$M_{0,3} * N_{3,1}$
thread _{1,0}	$M_{1,0} * N_{0,0}$	$M_{1,1} * N_{1,0}$	$M_{1,2} * N_{2,0}$	$M_{1,3} * N_{3,0}$
thread _{1,1}	$M_{1,0} * N_{0,1}$	$M_{1,1} * N_{1,1}$	$M_{1,2} * N_{2,1}$	$M_{1,3} * N_{3,1}$

FIGURE 5.6 Global memory accesses performed by threads in *block*_{0,0}.

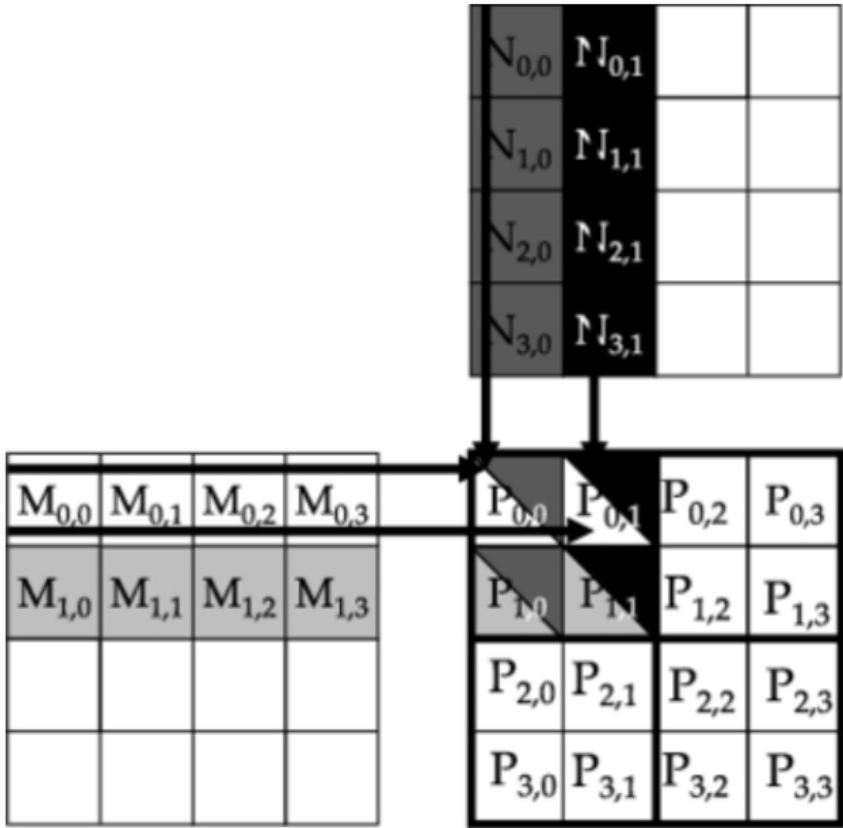


FIGURE 5.5 A small example of matrix multiplication. For brevity, We show $M[y*Width+x]$, $N[y*Width+x]$, and $P[y*Width+x]$ as $M_{y,x}$, $N_{y,x}$, and $P_{y,x}$, respectively.

Tiling To Reduce Global Memory Use

Courtesy: PMPP Textbook

- Figure shows 2x2 tiles. If rows/columns are loaded once per tile, this cuts global memory loads in half
- Threads in the same block use shared memory to store shared memory locations.
- All threads processing an output matrix tile can be assigned to the same thread block
- With larger tiles ($\text{TILE_WIDTH} > 2$), global memory loads are cut by a factor of TILE_WIDTH .
- Example: 32x32 tiles, memory bandwidth cut by a factor of 32.

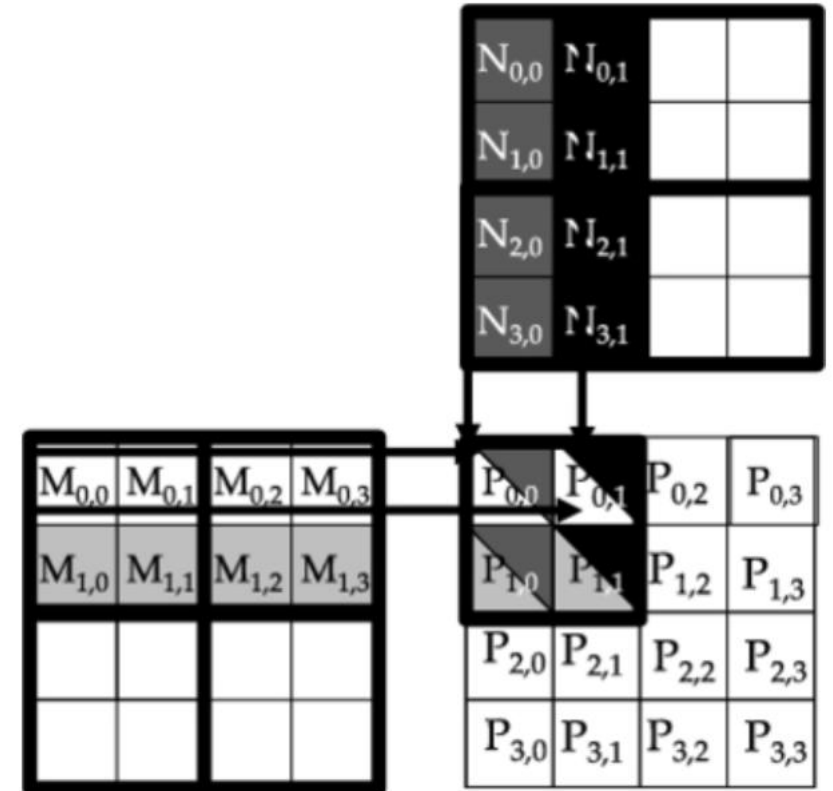


FIGURE 5.7 Tiling M and N to utilize shared memory.

Execution Phases for Tiled GEMM

Courtesy: PMPP Textbook

- Dot product in MM is divided into phases. In each phase, threads in the tile collaborate to load part of the input matrices and perform part of the dot product

- **Phase 0:**

- $M(0,0)$ - $M(1,1)$ loaded to $Mds(0,0)$ - $Mds(1,1)$
- $N(0,0)$ - $N(1,1)$ loaded to Nds
- Dot product of corresponding values in Mds and Nds accumulated to $Pvalue$

- **Phase 1:** Same for the tile from $M(0,2)$ - $(1,3)$ and $N(2,0)$ - $(3,1)$

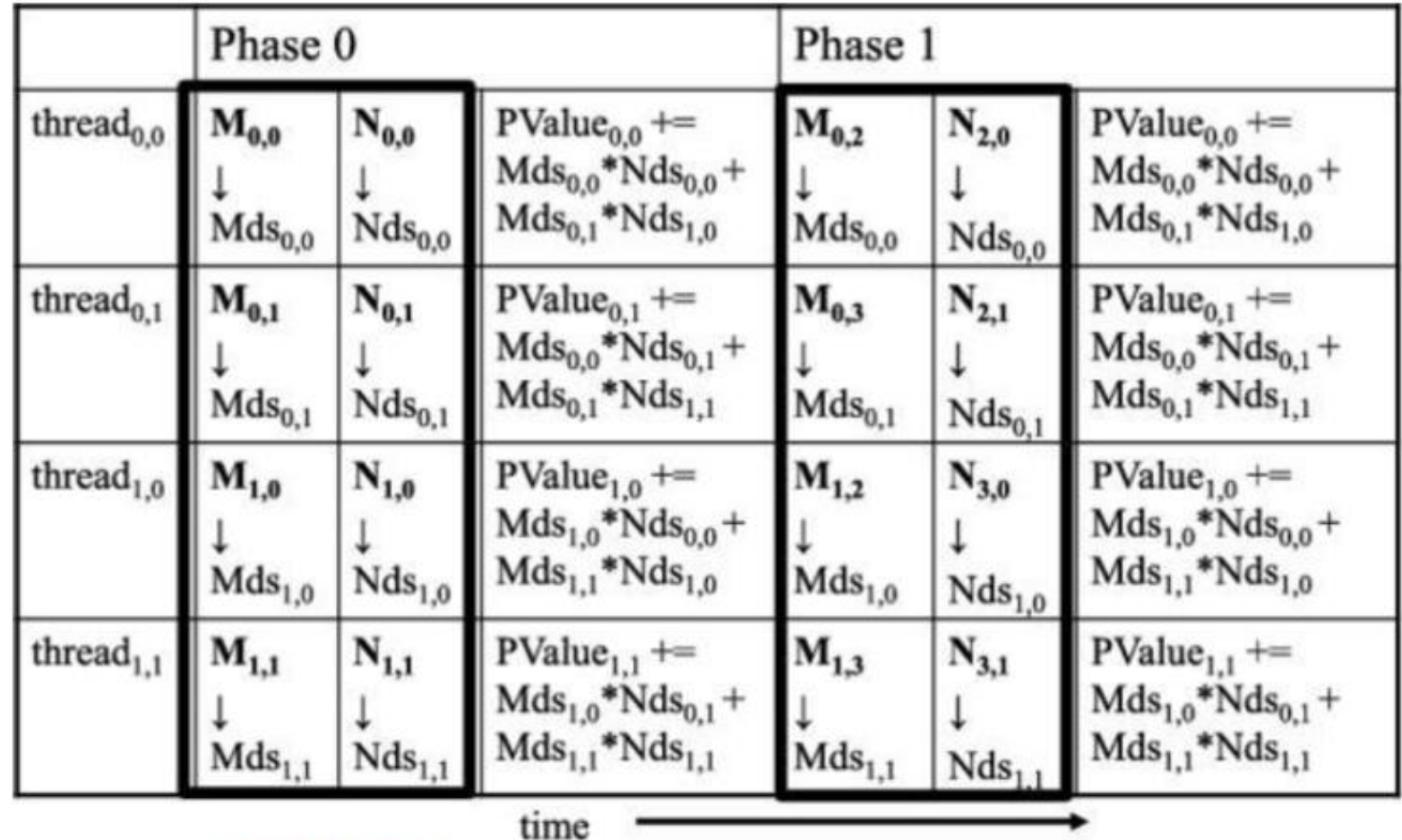


FIGURE 5.8 Execution phases of a tiled matrix multiplication.

Matrix Multiplication Kernel: Version 2

Courtesy: PMPP Textbook

- If input matrices have dimension Width, and tile has dimension TILE_WIDTH then Width/TILE_WIDTH phases are needed

Mds and Nds are in shared memory, shared across all threads in a block

Identify row, col of output matrix to compute

Needed to ensure all shared values are read before using in dot product

Needed to ensure all partial dot products done before reusing Mds, Nds

```
01  #define TILE_WIDTH 16
02  __global__ void matrixMulKernel(float* M, float* N, float* P, int width) {
03
04      __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
05      __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
06
07      int bx = blockIdx.x;  int by = blockIdx.y;
08      int tx = threadIdx.x; int ty = threadIdx.y;
09
10      // Identify the row and column of the P element to work on
11      int Row = by * TILE_WIDTH + ty;
12      int Col = bx * TILE_WIDTH + tx;
13
14      // Loop over the M and N tiles required to compute P element
15      float Pvalue = 0;
16      for (int ph = 0; ph < width/TILE_WIDTH; ++ph) {
17
18          // Collaborative loading of M and N tiles into shared memory
19          Mds[ty][tx] = M[Row*width + ph*TILE_WIDTH + tx];
20          Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*width + Col];
21          __syncthreads();
22
23          for (int k = 0; k < TILE_WIDTH; ++k) {
24              Pvalue += Mds[ty][k] * Nds[k][tx];
25          }
26          __syncthreads();
27
28      }
29      P[Row*width + Col] = Pvalue;
30
31 }
```

FIGURE 5.9 A tiled Matrix Multiplication Kernel using shared memory.

Matrix Multiplication Kernel: Version 2

Courtesy: PMPP Textbook

How to compute tile index per phase?

- In each phase ph , each thread loads elements from $M[row*width + ph*TILE_WIDTH + threadIdx.x]$ and $N[(ph*TILE_WIDTH + threadIdx.y) * width + col]$
- After all threads collaborate to load the tile, all perform partial dot products and accumulate into $Pvalue$
- First `__syncthreads()` is to handle read-after-write dependence (true dependence). All threads need to write to shared memory before any can use the shared values
- Second `__syncthreads()` is for a write-after-read (false) dependence since we are reusing Mds and Nds

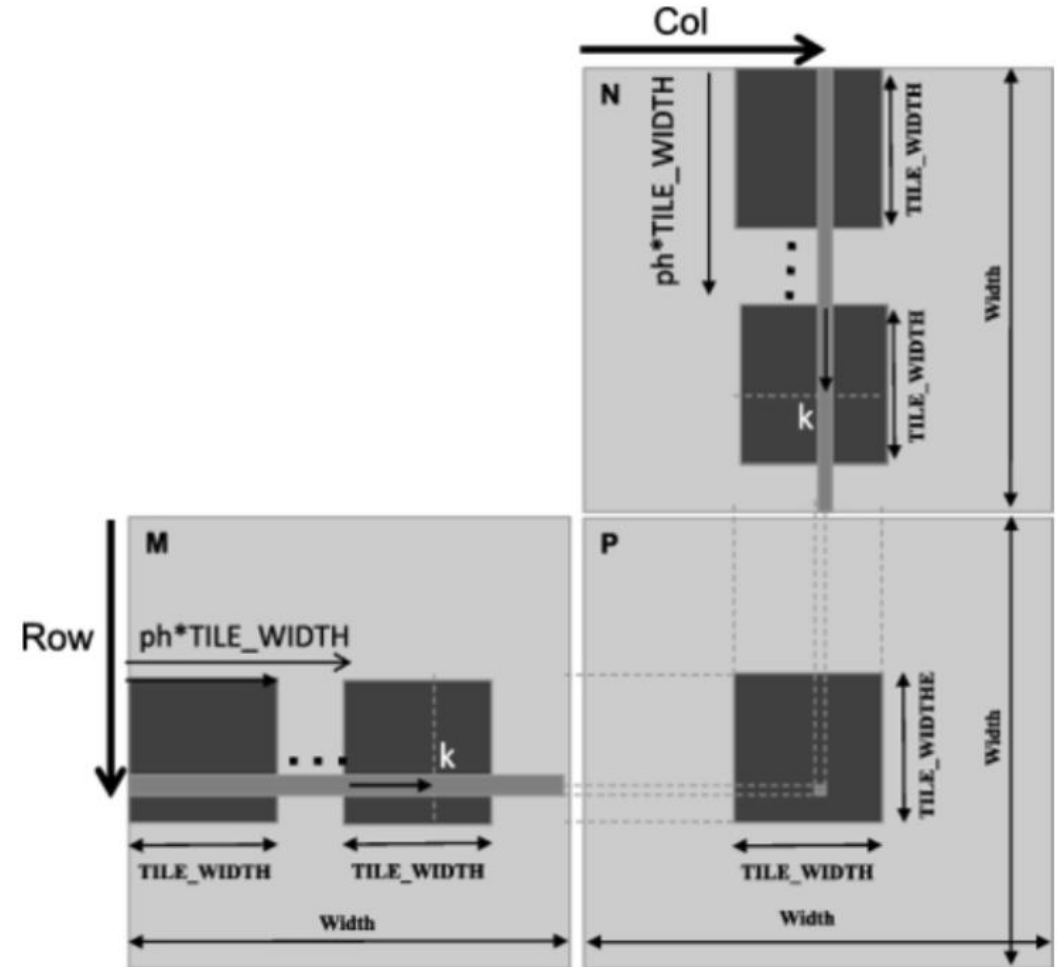
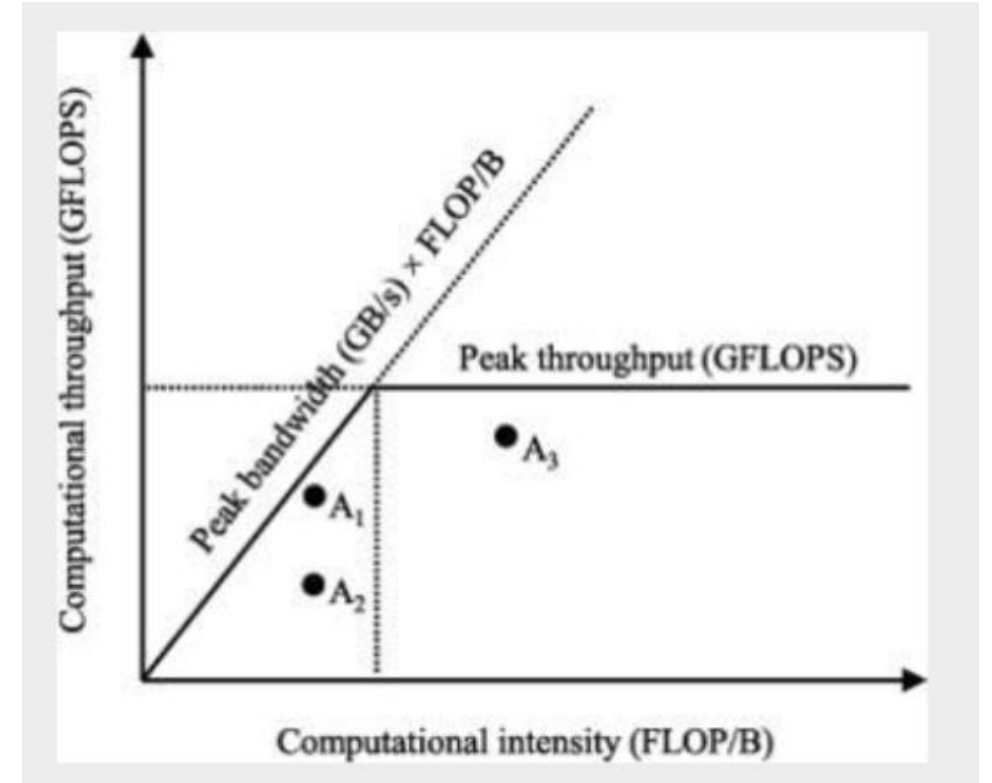


FIGURE 5.10 Calculation of the matrix indices in tiled multiplication.

Analysis of Tiled GEMM (V2)

Courtesy: PMPP Textbook

- GEMM V1 had computational intensity of 0.25 FLOP/B
- Using 32x32 tiles, GEMM V2 reduces memory accesses by a factor of 32 so it increases computational intensity to 8 FLOP/B
- Closer to H100's threshold of 20 FLOP/B but still memory-bound
- For H100 GPU with a global memory bandwidth of 3.35 TB/s, throughput now is
$$(3.35 \text{ TB/s}) \times (8 \text{ FLOP/B}) = 26.8 \text{ TFLOPS}$$
- Much higher than 0.84 TFLOPS of V1 but lower than H100's peak throughput of ~67 TFLOPS



Boundary Checking

- Special handling if Width is not a multiple of TILE_WIDTH

Courtesy: PMPP Textbook

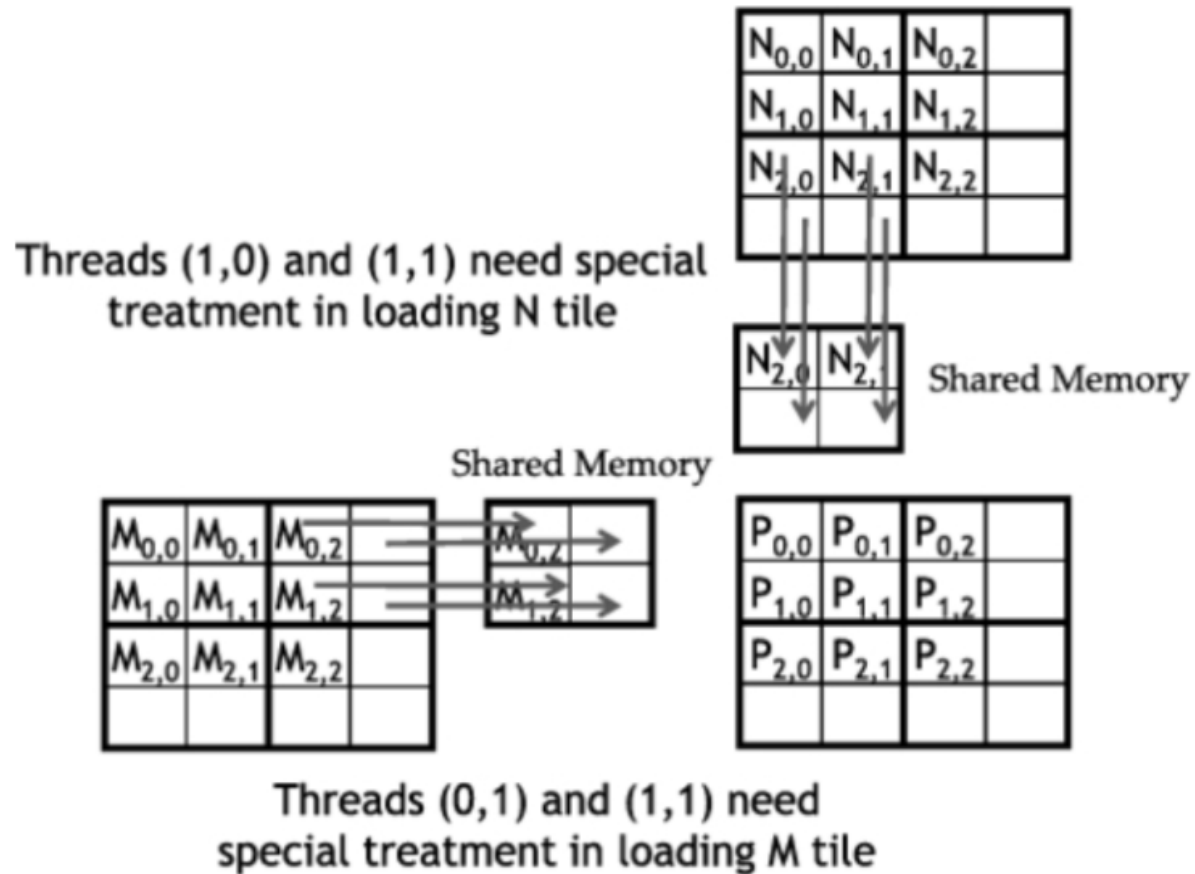


FIGURE 5.11 Loading input matrix elements that are close to the edge – phase 1 of $block_{0,0}$.

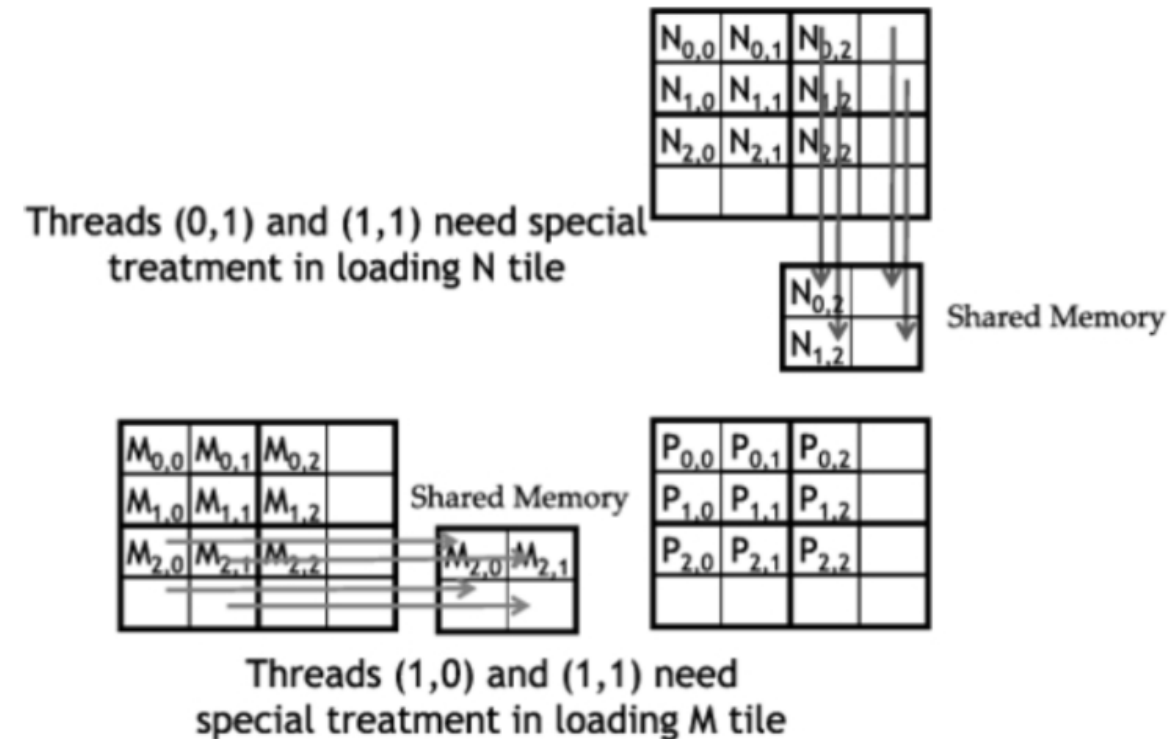


FIGURE 5.12 Loading input elements during phase 0 of $block_{1,1}$.

MM Kernel: Version 2 (w/Boundary Checks)

Courtesy: PMPP Textbook

- Load 0 into Mds or Nds if beyond boundary

```
14 // Loop over the M and N tiles required to compute P element
15 float Pvalue = 0;
16 for (int ph = 0; ph < ceil(width/(float)TILE_WIDTH); ++ph) {
17
18     // Collaborative loading of M and N tiles into shared memory
19     if ((Row < width) && (ph*TILE_WIDTH+tx) < width)
20         Mds[ty][tx] = M[Row*width + ph*TILE_WIDTH + tx];
21     else Mds[ty][tx] = 0.0f;
22     if ((ph*TILE_WIDTH+ty) < width && Col < width)
23         Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*width + Col];
24     else Nds[ty][tx] = 0.0f;
25     __syncthreads();
26
27     for (int k = 0; k < TILE_WIDTH; ++k) {
28         Pvalue += Mds[ty][k] * Nds[k][tx];
29     }
30     __syncthreads();
31 }
32 if (Row < width) && (Col < width)
33     P[Row*width + Col] = Pvalue;
```

FIGURE 5.13 Tiled matrix multiplication kernel with boundary condition checks.

Shared Memory Impact on Occupancy

- Similar to register usage, shared memory usage can also limit number of threads assigned to an SM
- Example: H100 GPU can have up to 228 KB of shared memory per SM, and a maximum of 2048 threads per SM. For all 2048 thread slots to be used, a thread block should not use more than an average of $(228 \text{ KB}) / (2048 \text{ threads}) = 114 \text{ B/thread}$.
- Example: RTX2000 GPU can have up to 99 KB of shared memory per SM, and a maximum of 1536 threads per SM. For all thread slots to be used, a thread block should not use more than an average of $(99 \text{ KB}) / (1536 \text{ threads}) = 66 \text{ B/thread}$.
- Does GEMM V2 exceed these thresholds?
 - Assume each block has $\text{TILE_WIDTH} \times \text{TILE_WIDTH}$ threads.
 - Each block uses $\text{TILE_WIDTH}^2 \times (4\text{B})$ for each of Matrix Mds and Nds, so the total shared memory per block is 8 TILE_WIDTH^2 bytes
 - Shared memory per thread = $(\text{Shared Memory per Block}) / (\# \text{ Threads per Block}) = 8\text{B/thread}$ (lower than thresholds, so no extra limit on occupancy)

Shared Memory Impact on Occupancy

- Assume a different kernel with 256 threads/block and 38 KB/block of shared memory needed. Shared memory per thread is $38\text{K}/256 = 152 \text{ B/thread}$, higher than thresholds
- In H100, this limits number of threads per SM to $(228\text{KB})/(152 \text{ B/thread}) = 1536 \text{ threads}$ (75% occupancy)
- `cudaGetDeviceProperties()` can be used to determine amount of shared memory per SM: `devProp.sharedMemPerBlock`
- This can be used to adjust the sizes of shared memory used per block, but requires dynamic sizing

Dynamic Shared Memory Use

Courtesy: PMPP Textbook

```
01  #define TILE_WIDTH 32
02  __global__ void matrixMulKernel(float* M, float* N, float* P, int Width,
                                unsigned Mdz_sz, unsigned Nds_sz) {
03
04      extern __shared__ char float Mds_Nds[];
05
06      float *Mds = (float *) Mds_Nds;
07      float *Nds = (float *) Mds_Nds + Mds_sz;
```

FIGURE 5.14 Tiled matrix multiplication kernel with dynamically sized shared memory usage.

- Dynamic size can be computed before kernel call

```
size_t size = ...;
```

```
matrixMulKernel <<< dimGrid, dimBlock, size >>> (Md, Nd, Pd, Width, size/2, size/2);
```

- Both Mds and Nds are combined into one shared storage with `extern __shared__` declaration. Each matrix uses half the allocated storage. (Mds_size and Nds_size is both size/2)
- The kernel should use linearized index to access Mds and Nds – instead of `Mds[ty][tx]` use `Mds[ty*TILE_WIDTH+tx]`

Performance Optimizations

Global Memory Access Coalescing 1

Courtesy: PMPP Textbook

- **DRAM Burst:** Each time a DRAM location is accessed, a range of consecutive locations that include the requested location are also accessed (usually 32B or 64B).
- GPUs include SRAM-based L1 and L2 on-chip caches.
 - When data has already been fetched from the DRAM and placed in caches, accessing consecutive memory locations within a *cache line* (typically 128B) helps take advantage of spatial locality.
- All threads in a warp execute the same instruction.
 - For load instructions, the hardware detects whether warp threads access consecutive global memory locations in the same DRAM burst and/or cache line
 - In that case, hardware coalesces all these accesses into a consolidated access $\Rightarrow$ fewer total memory accesses

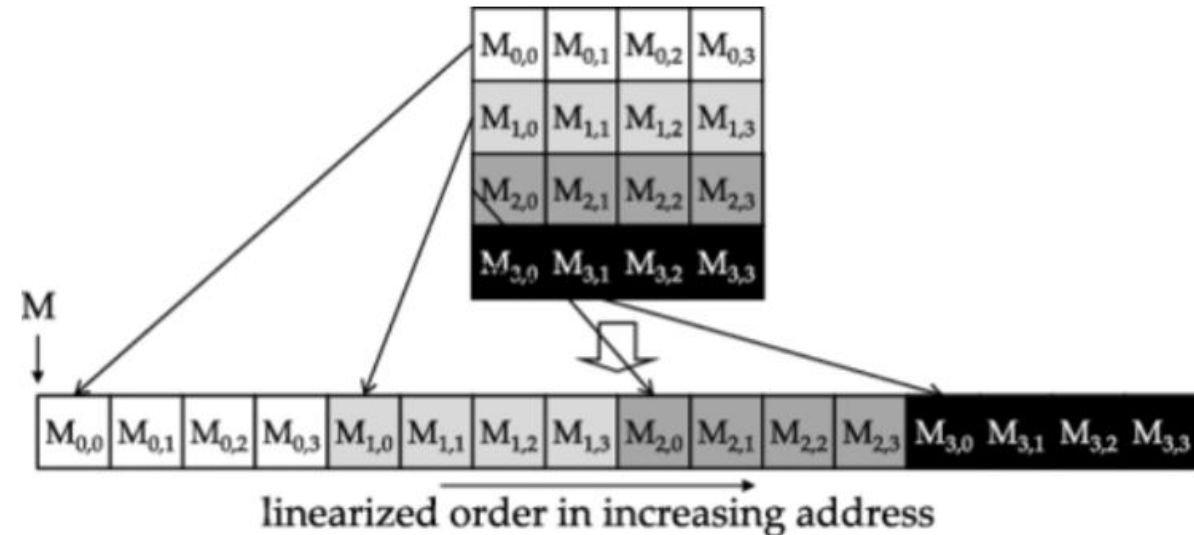


FIGURE 6.1 Placing matrix elements into a linear array based on row major order.

Global Memory Access Coalescing 2

Courtesy: PMPP Textbook

- Assume array N is the second array in GEMM kernel. Figure shows access pattern by different threads.

Code

```
unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
if (row < Width && col < Width) {
    float Pvalue = 0.0f;
    for(unsigned int k = 0; k < Width; ++k) {
        Pvalue += M[row*Width + k]*N[k*Width + col];
    }
    P[row*Width + col] = Pvalue;
}
```

- Index into N is $k \cdot \text{Width} + \text{col}$
- Both k and Width are the same for all threads in a warp
- $\text{col} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$ so consecutive threads with consecutive threadIdx.x values will access consecutive columns

- Conclusion: Threads in the same warp access consecutive locations in N, so accesses can be coalesced.

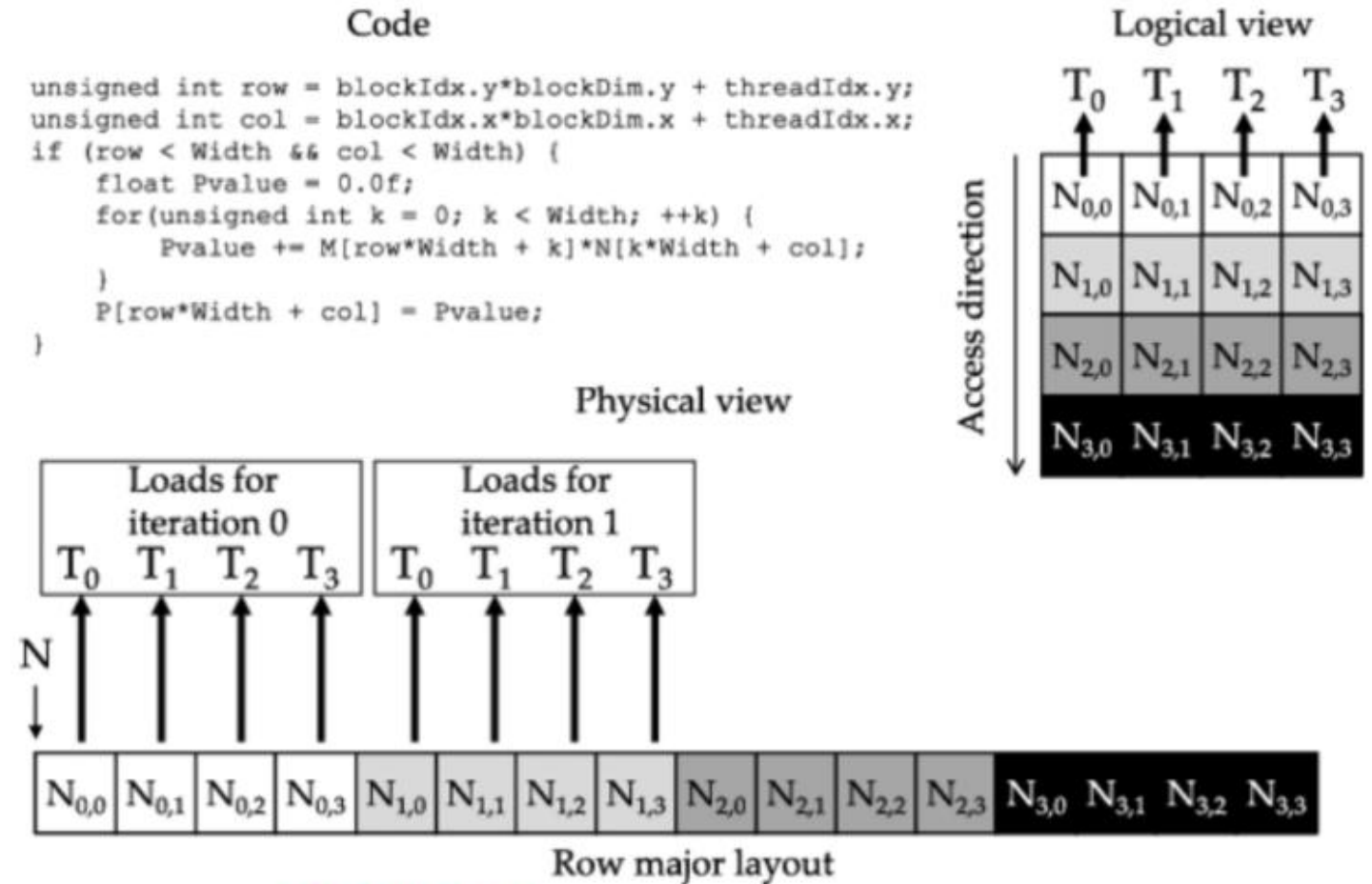


FIGURE 6.2 A coalesced access pattern.

Global Memory Access Coalescing 3

Courtesy: PMPP Textbook

- Assume N is stored in column-major order.
 - Index into N is $\text{col} * \text{Width} + k$
 - Both k and Width are the same for all threads in a warp
 - $\text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ so consecutive threads with consecutive threadIdx.x values will still access consecutive col values
 - But consecutive col values are stored *Width* values apart in memory, so accesses **cannot be coalesced**.
- We need to change the order of thread access or data layout to take advantage of global memory access coalescing

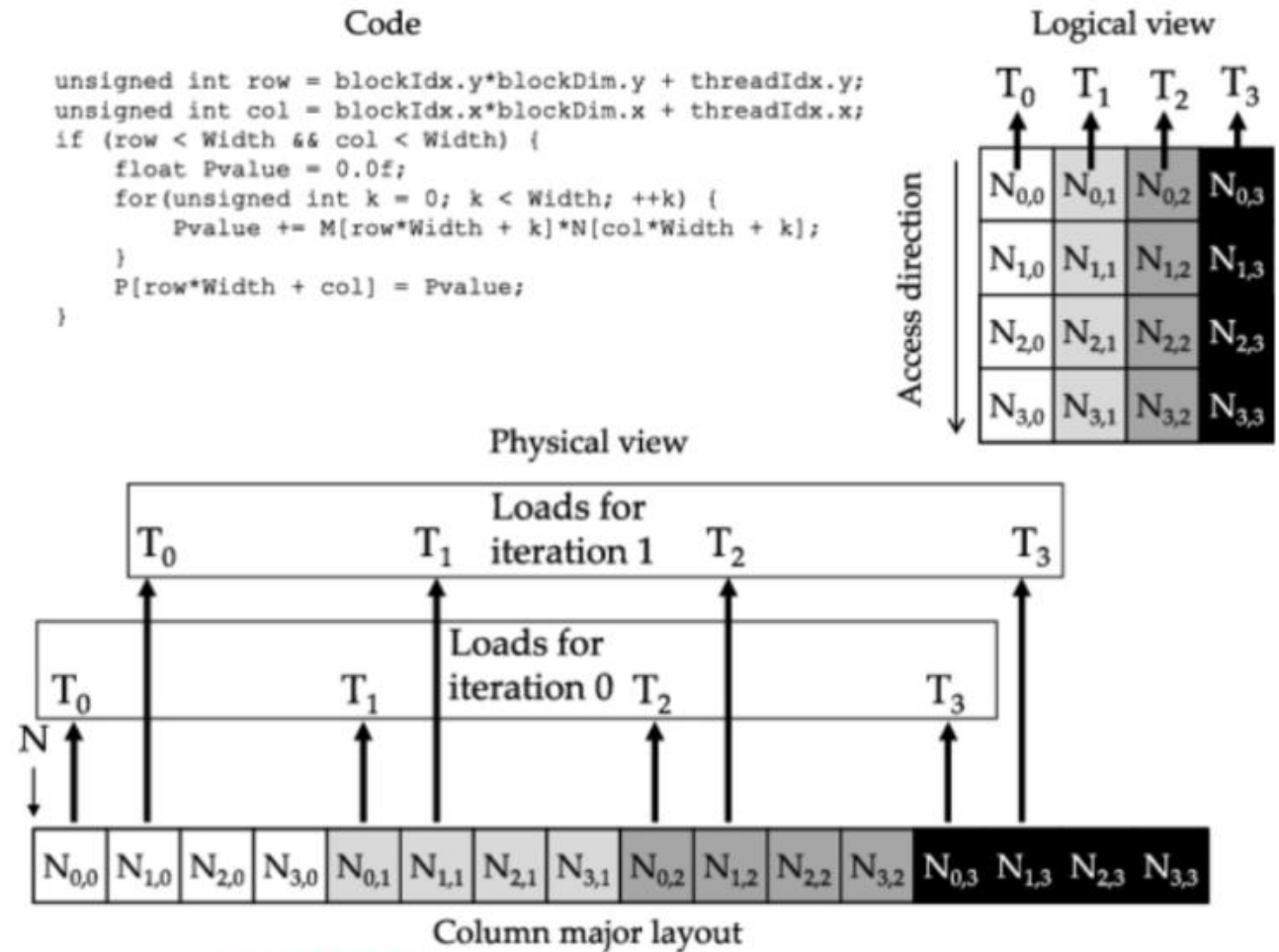


FIGURE 6.3 An un-coalesced access pattern.

Global Memory Access Coalescing 4

Courtesy: PMPP Textbook

- One solution: **Corner turning**
- From the GEMM V2 kernel, accesses to M are kept the same
- Accesses to N are reordered so each thread accesses the left edge of the tile in phase 0 instead of the top edge. Other phases are similarly adjusted
 - Exchange threadIdx.x and threadIdx.y when computing the linearized address to access N
 - With column-major N, consecutive elements in the same column are adjacent in memory. So consecutive threads load elements that can be coalesced.

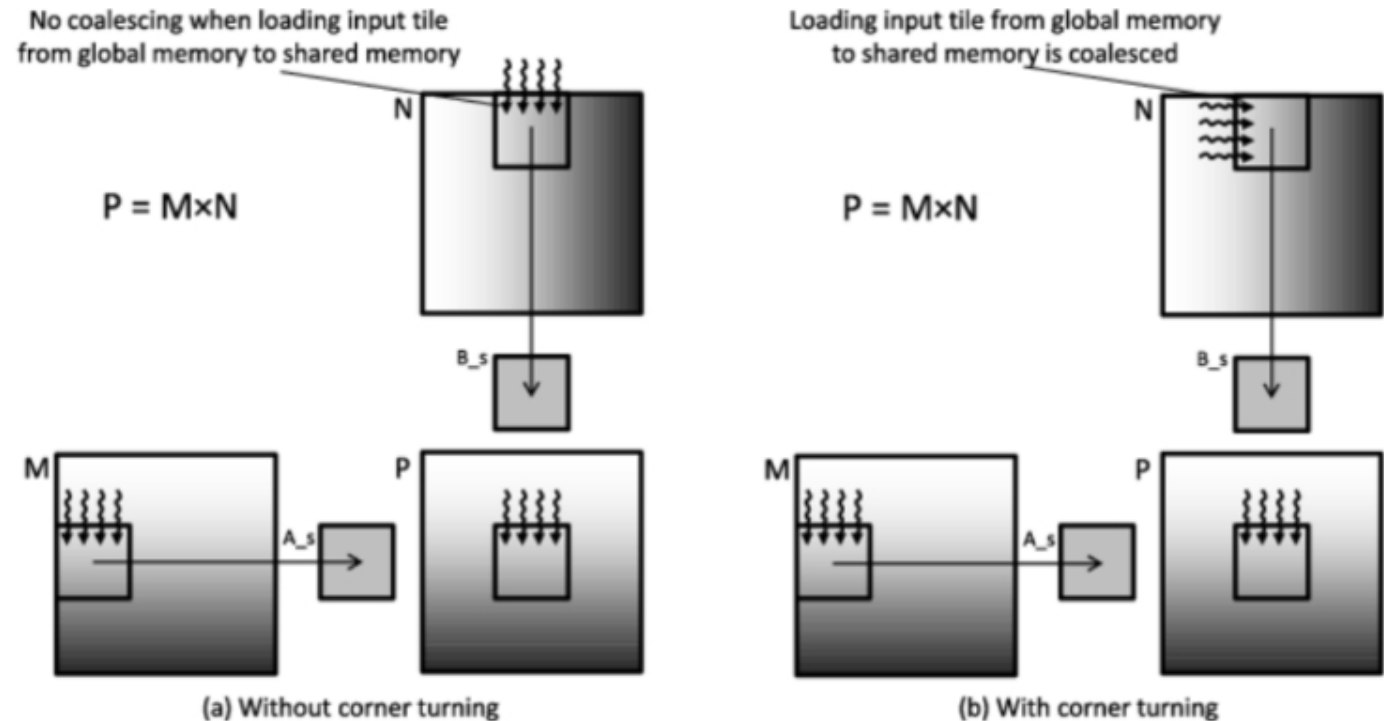


FIGURE 6.4 Applying corner turning to coalesce accesses to matrix N which is stored in column-major layout.

Hiding Memory Latency

Courtesy: PMPP Textbook

- Can be done via parallel memory accesses, so the latency per request is hidden by other parallel accesses
- DRAM bandwidth depends on frequency (with double data rate), number of channels, banks per channel, and data width for each channel
- Example: Single channel 64-bit DDR5-6400 bus with a clock frequency of 3.2 GHz has a bandwidth of $8 \text{ B} \times 2 \times 3.2 \text{ GHz} = 51.2 \text{ GB/sec}$. (much smaller than needed by GPU)
- HBM3E supports 4.9 GHz frequency and 16 64-bit channels, resulting in $\sim 1 \text{ TB/s}$ bandwidth

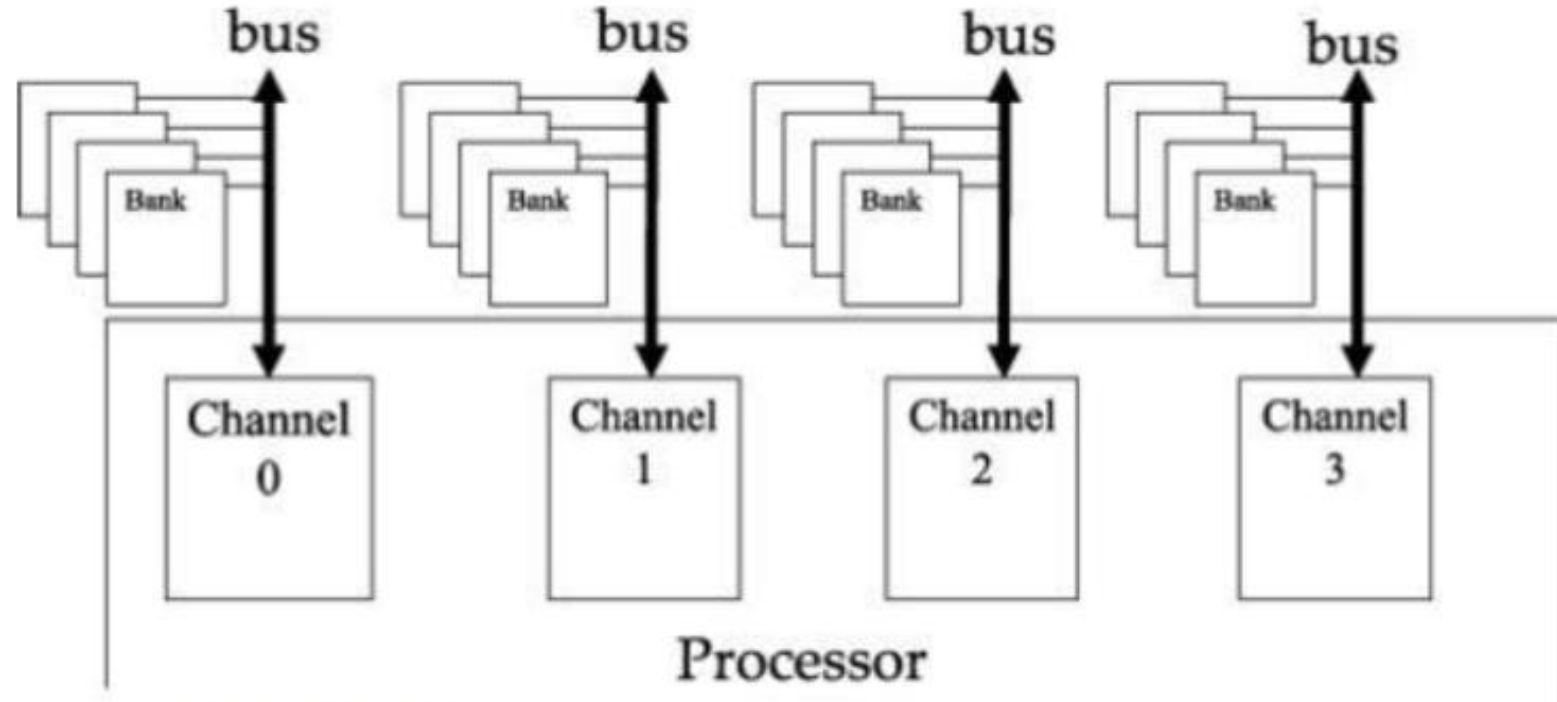


FIGURE 6.7 Channels and banks in DRAM systems.

Improving Memory Bandwidth Utilization

Courtesy: PMPP Textbook

- For each channel, #banks is determined by the number needed to fully bandwidth.
- Each bank requires long latency between consecutive accesses (unless we have row buffer hits)
- Accesses to different banks can use the bus at different times, therefore hiding the latency of each access.
- With two banks, we can potentially double channel bandwidth utilization.
- If the ratio of the cell array access latency and data transfer time is R , we need to have at least $R + 1$ banks to fully utilize bandwidth.

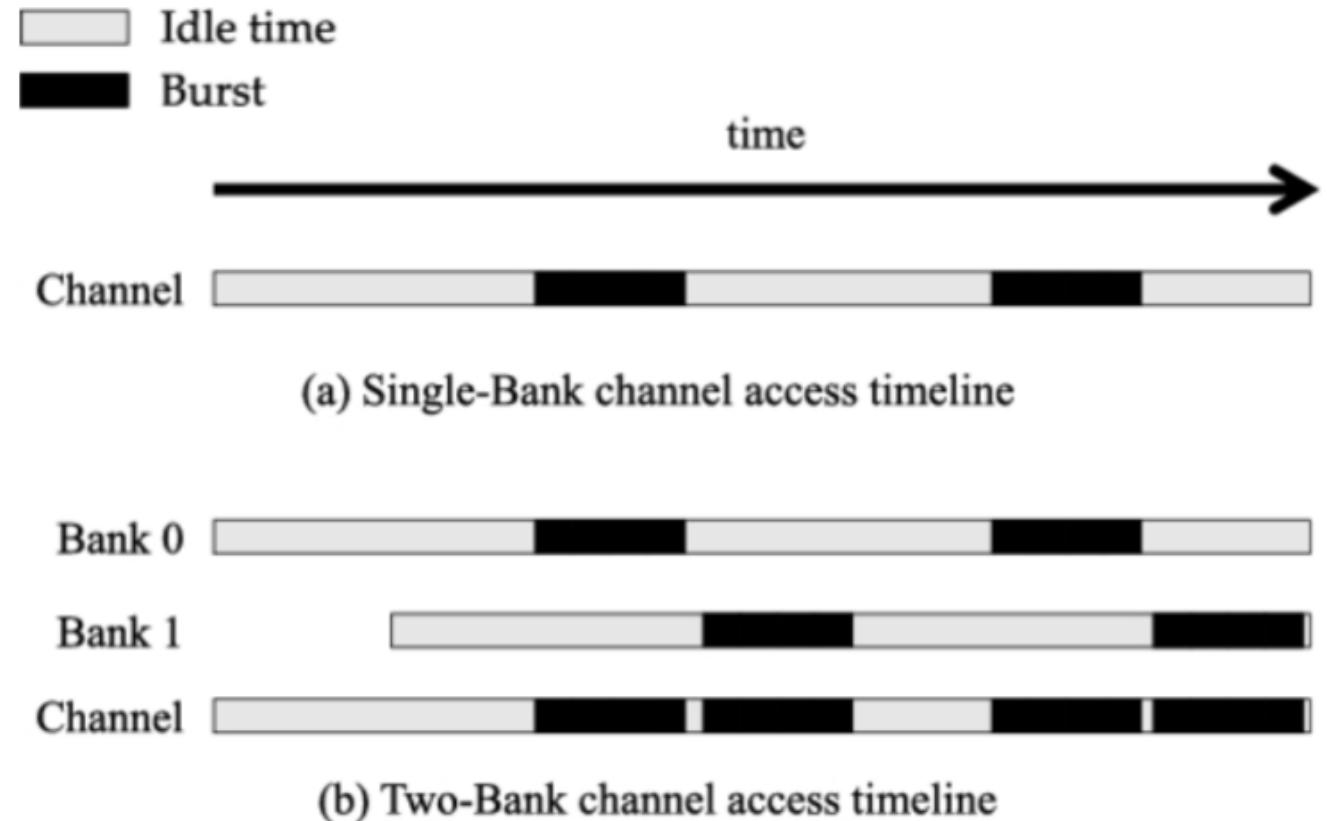


FIGURE 6.8 Banking improves the utilization of data transfer bandwidth of a channel.

Improving Memory Bandwidth Utilization

Courtesy: PMPP Textbook

- Assume a small burst size of two FP32 elements (8B).
- Assume hardware address mapping distributes accesses across channels and banks as follows (interleaved data distribution):
 - The first 8B of the array (M[0] and M[1]) are stored in Bank 0 of Channel 0,
 - Next 8B (M[2] and M[3]) Bank 0 of Channel 1
 - Next 8B (M[4] and M[5]) Bank 0 of Channel 2
 - M[6] and M[7] Bank 0 of Channel 3.
 - Then M[8] and M[9] wrap back to bank 1 Channel 0...
- Interleaved distribution ensures all channels/banks are utilized (in this example, even 16 elements can saturate BW).

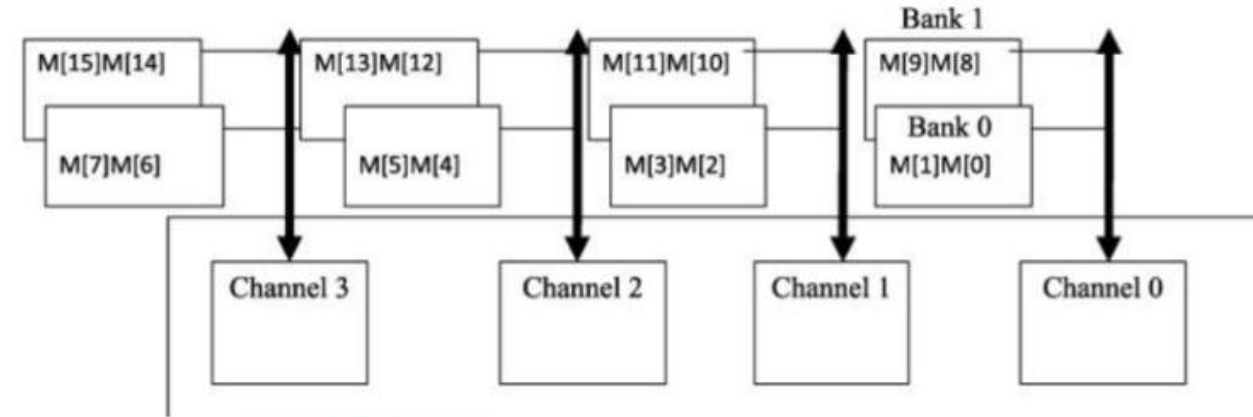


FIGURE 6.9 Distributing array elements into channels and banks.

Example: GEMM

Courtesy: PMPP Textbook

- Phase 0 coalesces accesses to Channels 0 and 2
- Phase 1 coalesces accesses to Channels 1 and 3
- This changes with larger tiles, larger bursts, more channels

Tiles loaded by	Block 0,0	Block 0,1	Block 1,0	Block 1,1
Phase 0 (2D index)	M[0][0], M[0][1], M[1][0], M[1][1]	M[0][0], M[0][1], M[1][0], M[1][1]	M[2][0], M[2][1], M[3][0], M[3][1]	M[2][0], M[2][1], M[3][0], M[3][1]
Phase 0 (linearized index)	M[0], M[1], M[4], M[5]	M[0], M[1], M[4], M[5]	M[8], M[9], M[12], M[13]	M[8], M[9], M[12], M[13]
Phase 1 (2D index)	M[0][2], M[0][3], M[1][2], M[1][3]	M[0][2], M[0][3], M[1][2], M[1][3]	M[2][2], M[2][3], M[3][2], M[3][3]	M[2][2], M[2][3], M[3][2], M[3][3]
Phase 1 (linearized index)	M[2], M[3], M[6], M[7]	M[2], M[3], M[6], M[7]	M[10], M[11], M[14], M[15]	M[10], M[11], M[14], M[15]

FIGURE 6.11 M elements loaded by thread blocks in each phase.

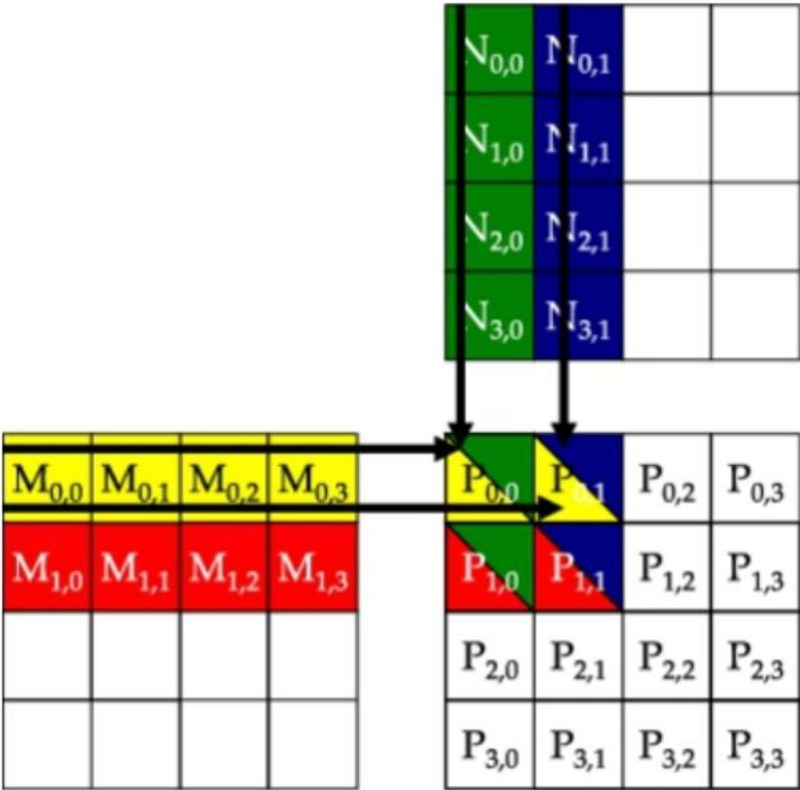


FIGURE 6.10 A small example of matrix multiplication (replicated from Fig. 5.5).

Vector Loads and Stores

Courtesy: PMPP Textbook

- Better BW utilization if each thread access a larger amount of continuous data in a single instruction: vector loads and stores
- This kernel launches one fourth of the threads in the original vector add kernel, and each thread is performing 4 adds.
- When a pointer to a float4 datatype is dereferenced, the compiler converts the access to a vector load or store that accesses the entire 16 B object.
- Loads and stores will be performed as vector loads/stores to improve BW utilization

```
01  __global__ void vecadd_kernel(float* x, float* y, float* z, int N) {  
02      int i = blockDim.x*blockIdx.x + threadIdx.x;  
03      float4 x4 = ((float4*)x)[i];  
04      float4 y4 = ((float4*)y)[i];  
05      float4 z4;  
06      z4.x = x4.x + y4.x;  
07      z4.y = x4.y + y4.y;  
08      z4.z = x4.z + y4.z;  
09      z4.w = x4.w + y4.w;  
10      ((float4*)z)[i] = z4;  
11  }
```

FIGURE 6.12 A vector addition kernel that uses vector loads.

Shared Memory Bank Conflicts

- A typical shared memory structure on GPUs has an access latency of a few clock cycles and consists of 32 banks. Each bank is 32 bits (4B) wide.
- When a GPU thread accesses a 32-bit INT or FP value from shared memory, the thread alone utilizes the output of an entire bank.
- When threads in the same warp access different values from shared memory, it is desirable for each thread to access a different memory bank so that the values can be served in parallel.
- To maximize shared memory BW, shared memory is organized such that consecutive 4B chunks are placed in different banks in an interleaved fashion (similar to DRAM bank interleaving)
- if the accesses by threads in the same warp are less adjacent to each other, they might hit the same bank at the same time, which results in a **bank conflict**.
- Since accesses to the same bank need to be served one at a time, a bank conflict is detected by hardware and accesses are split into multiple conflict-free accesses

Avoiding Shared Memory Bank Conflicts

- Example for bank conflict: Writes to output matrix in GEMM V2 with 32x32 tiles:

```
__shared__ float a[TILE_DIM][TILE_DIM]; // TILE_DIM = 32  
a[threadIdx.x][threadIdx.y] = ...;
```

- Threads in the same warp will have the same threadIdx.y value and consecutive threadIdx.x values. So consecutive threads will write to values that are 32 4B values apart (that is, to the same bank). Thread 0 writes to a[0][0], thread 1 writes to a[1][0] etc.
- If a[0][0] is allocated to bank 0, all other a[n][0] will also be allocated to bank 0: bank conflict which leads to serialization
- To fix, trade off capacity to avoid bank conflict:

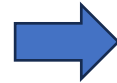
```
__shared__ float a[TILE_DIM][TILE_DIM + 1]; // TILE_DIM = 32  
a[threadIdx.x][threadIdx.y] = ...;
```

Number of columns in each a row is TILE_DIM+1 instead of TILE_DIM, so accesses go to different banks. Causes more complexity in computing index (add and multiply instead of shift)

Thread Coarsening

- Most of previous kernels performed computations at the finest granularity. But that can be expensive in case of synchronization or other parallelization overheads (redundant memory accesses, redundant computations).
- Could be avoided via thread coarsening where fewer threads perform more work each.

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
foo(i);
```

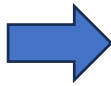


```
unsigned int iStart = 4*(blockIdx.x*blockDim.x + threadIdx.x);  
for(unsigned int c = 0; c < 4; ++c) {  
    unsigned int i = iStart + c;  
    foo(i);  
}
```

- Guidelines for thread coarsening:
 - Only perform when necessary, i.e. when parallelization overheads outweigh benefit.
 - Not too much coarsening to result in under-utilization of hardware resources (think of waves of blocks)
 - Avoid increasing resource use so it reduces occupancy (think of registers or shared memory use per thread).

Loop Unrolling

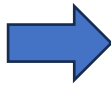
- Defined as replicating the body of the loop by some factor and reducing the number of loop iterations by the same factor.
 - Results in fewer loop iterations, so fewer end of loop branches

<pre>for(unsigned int i = 0; i < 16; ++i) { A(i) B(i) }</pre>		<pre>for(unsigned int i = 0; i < 16; i += 4) { A(i) B(i) A(i + 1) B(i + 1) A(i + 2) B(i + 2) A(i + 3) B(i + 3) }</pre>
--	---	---

Can be done with compiler pragma `#pragma unroll 4`

Loop Unrolling

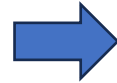
- Exposes more independent instructions that can be reordered to avoid stalling

<pre>for(unsigned int i = 0; i < 16; i += 4) { A(i) B(i) A(i + 1) B(i + 1) A(i + 2) B(i + 2) A(i + 3) B(i + 3) }</pre>		<pre>for(unsigned int i = 0; i < 16; i += 4) { A(i) A(i + 1) A(i + 2) A(i + 3) B(i) B(i + 1) B(i + 2) B(i + 3) }</pre>
---	---	---

Double Buffering

- If shared memory space is available, we can use double buffering to reduce synchronization overhead caused by false (write after read) dependence.

```
for(unsigned int i = 0; i < N; ++i) {  
    ... = buffer[anotherThreadID];  
    __syncthreads();  
    buffer[myThreadID] = ...;  
    __syncthreads();  
}
```



```
for(unsigned int i = 0; i < N; ++i) {  
    ... = inBuffer[anotherThreadID];  
    outBuffer[myThreadID] = ...;  
    __syncthreads();  
    swap(inBuffer, outBuffer);  
}
```

Optimizations Checklist

Category	Optimization	Benefit to Compute Cores	Benefit to Memory	Strategies
Compute utilization	Occupancy tuning	More work to hide pipeline latency	More parallel memory accesses to hide DRAM latency	Tune the usage of SM resources such as threads per block, shared memory per block, and registers per thread
	Loop unrolling	Fewer branch instructions and more independent instruction sequences with fewer stalls	May enable promoting local arrays to registers to reduce global memory traffic	Performed automatically by the compiler Use loops with constant bounds where possible to facilitate the compiler's job
	Reducing control divergence	High SIMD efficiency (fewer idle cores during SIMD execution)	-	Rearrange the assignment of threads to work and/or data Rearranging the layout of the data
Memory utilization	Using coalesceable global memory accesses	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic and better utilization of bursts/cache-lines	Rearranging the mapping of threads to data Rearranging the layout of the data (e.g., matrix transposition, converting an array-of-structures to a structure-of-arrays) Transfer data between global memory and shared memory in a coalesceable manner and perform irregular accesses in shared memory (e.g., corner turning, packing)
	Shared memory tiling	Fewer pipeline stalls waiting for global memory accesses	Less global memory traffic	Place data that is reused within a block in shared memory so that it is transferred between global memory and the SM only once
	Register tiling	Fewer pipeline stalls waiting for shared memory accesses	Less shared memory traffic	Place data that is reused within a warp or thread in registers so that it is transferred between shared memory and registers only once
	Vector loads and stores	Fewer instructions executed to manage memory accesses	More parallel memory accesses to hide DRAM latency	Assign each thread to load 16 B chunks of contiguous data
	Avoiding shared memory bank conflicts	Fewer pipeline stalls waiting for shared memory accesses	Less shared memory traffic and better utilization of shared memory banks	Rearranging the mapping of threads to data Rearranging the layout of the data (e.g., padding)
	Privatization (covered later)	Fewer pipeline stalls waiting for atomic updates	Less contention and serialization of atomic updates	Apply partial updates to a private copy of the data then update the public copy when done
Synchron-ization latency	Warp-level primitives	Reduce block-wide barrier synchronizations	Less global and shared memory traffic	Perform operations requiring barrier synchronization at the warp-level, then consolidate warp-level results at the block-level
	Double buffering	Eliminates barriers that enforce false dependences	-	Eliminate false (write-after-read) dependences by using different buffers for the writes and the preceding reads
General	Thread coarsening	Depends on the overhead of parallelization	Depends on the overhead of parallelization	Assign multiple units of parallelism to each thread in order to reduce the overhead of parallelization when it is incurred unnecessarily

Courtesy: PMPP Textbook

Optimization Strategy

- Need to understand which resource is the performance bottleneck.
 - No need to optimize a non-critical resource
- Can be done iteratively:
 - Identify bottleneck
 - Optimize
 - Identify new bottleneck
 - Optimize etc.
- Need to understand hardware limits (e.g., roofline) to avoid further optimization after limit is reached.

Exercises (From Textbook)

Courtesy: PMPP Textbook

- For our tiled matrix-matrix multiplication kernel, if we use a 32×32 tile, what is the reduction of memory bandwidth usage for input matrices M and N ?
- Assume that a CUDA kernel is launched with 1000 thread blocks each of which has 512 threads. If a variable is declared as a local variable in the kernel, how many versions of the variable will be created through the lifetime of the execution of the kernel?
- In the previous question, if a variable is declared as a shared memory variable, how many versions of the variable will be created through the lifetime of the execution of the kernel?
- Consider performing a matrix multiplication of two input matrices with dimensions $N \times N$. How many times is each element in the input matrices requested from global memory when:
 - a. There is no tiling?
 - b. Tiles of size $T \times T$ are used?

Exercises (From Textbook)

Courtesy: PMPP Textbook

- A kernel performs 36 floating point operations and 7 32-bit global memory accesses per thread. For each of the following device properties, indicate whether this kernel is compute- or memory-bound.
 - a. Peak FLOPS = 200 GFLOPS, Peak Memory Bandwidth = 100 GB/s
 - b. Peak FLOPS = 300 GFLOPS, Peak Memory Bandwidth = 250 GB/s
- Consider a GPU with the following hardware limits: 2048 threads/SM, 32 blocks/SM, 64 K (65536) registers/SM, and 96 KB of shared memory/SM. For each of the following kernel characteristics, specify if the kernel can achieve full occupancy. If not, specify the limiting factor.
 - a. The kernel uses 64 threads/block, 27 registers/thread, and 4 KB of shared memory/SM.
 - b. The kernel uses 256 threads/block, 31 registers/thread, and 8 KB of shared memory/SM.

Exercises (from Textbook)

Courtesy: PMPP Textbook

- How many versions of the variable `i` are there?
- How many versions of the array `x[]` are there?
- How many versions of the variable `y_s` are there?
- How many versions of the array `b_s[]` are there?
- What is the amount of shared memory used per block (in B)?
- What is the floating point to global memory access ratio of the kernel (in OP/B)?

```
01  __global__ void foo_kernel(float* a, float* b) {
02      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
03      float x[4];
04      __shared__ float y_s;
05      __shared__ float b_s[128];
06      for(unsigned int j = 0; j < 4; ++j) {
07          x[j] = a[j*blockDim.x*gridDim.x + i];
08      }
09      if(threadIdx.x == 0) {
10          y_s = 7.4f;
11      }
12      b_s[threadIdx.x] = b[i];
13      __syncthreads();
14      b[i] = 2.5f*x[0] + 3.7f*x[1] + 6.3f*x[2] + 8.5f*x[3]
15          + y_s*b_s[threadIdx.x] + b_s[(threadIdx.x +
16      3)%128];
17  }
18  void foo(int* a_d, int* b_d) {
19      unsigned int \texttt{N} = 1024;
20      foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d);
21  }
```

Exercises (From Textbook)

Courtesy: PMPP Textbook

- Consider the following shared memory array and access to the array by a block with one warp (32 threads):

```
__shared__ float a[1024];  
a[threadIdx.x*stride] = ...;
```

Assume that $a[0]$ is placed in bank 0. For each of the following values of stride, specify the banks accessed by the warp and the number of bank conflicts on each bank (if any):

- a. stride = 32
- b. stride = 31
- c. stride = 24
- d. stride = 16
- e. stride = 12

Class Announcements

- **HW3: Due Monday Oct 5**
- **This week's reading:**
 - PMPP Chapters 5-6
- **No class next week. Exam 1 at the start of class in 2 weeks**
 - Details will be sent out on Piazza, so please sign up if you haven't already.
- **Next week's reading:**
 - PMPP Chapters 7-8